Graft: Time-Aware Pacing, Structural Grounding, and Context Compaction for Autonomous Software Engineering with Gemma 4

Frank Acquaye **Project Graft Team**
acquayefrank@gmail.com
Google - The Gemma 4 Developer Agent Paper Track

October 2026

Abstract

Autonomous software engineering agents powered by open-weight foundation models represent a vital frontier for verifiable, cost-effective, and offline code generation. However, evaluating these models in SWE-bench-style benchmarks under competition envelopes reveals operational hurdles that conventional synthetic benchmarks obscure: aggressive wall-clock timeouts (∼5.0 minutes per task across 120 tasks in 12 hours), finite context windows, and strict two-phase containerized verification. In this work, we present **Graft**, an autonomous software engineering agent built on **Gemma 4 31B W4A16** running on 4x NVIDIA L4 GPUs.

Through granular step-by-step trace analysis of early failure modes, we identify three critical pathologies common to open-weight agents: (1) **post-fix over-verification**, where an agent discovers and verifies a working fix but continues running redundant tests until hitting a hard session timeout; (2) **scratchpad contamination and path traversal errors**, where the agent attempts to write temporary reproduction scripts to restricted directories (`/tmp`) or contaminates the patch with untracked files; and (3) **lexical localization fatigue**, where linear grep commands exhaust turn and token budgets.

To overcome these pathologies, Graft introduces a **time-aware pacing architecture** featuring a strict stop-and-submit terminal protocol, workspace-isolated `build/` scratchpads, and symbol-level code graph grounding via precomputed AST and embedding graphs. Furthermore, we explore **hierarchical context compaction** via an isolated, read-only sub-agent (`code_analyzer`), empirically ablating its token-efficiency benefits against its coordination latency tax. On our 30-task development benchmark (`dev30`), Graft's paced single-agent configuration achieves a **53.3% resolution rate (16/30)**—an absolute improvement of +16.6% over the unconstrained baseline (36.7%)—while reducing mean agent latency by 9.1s and projecting a completion time of 7.64 hours for the 120-task competition window.

1 Introduction

Software engineering requires multi-hop reasoning, deep semantic code navigation, and precise syntactic code modification. While proprietary frontier models have demonstrated notable success on benchmarks such as SWE-bench, deploying open-weight foundation models—such as the **Gemma 4** family—within autonomous developer loops introduces distinct architectural and systems challenges:

1. **The Shared Wall-Clock Horizon**: In competitive SWE-bench evaluations on Kaggle, an agent must evaluate ∼120 hidden issues within a shared 12-hour global session. Accounting for container provisioning and editable wheel installs, each task can allocate at most **4.5 to 5.0 minutes** of active LLM reasoning time; exploratory rabbit holes directly starve subsequent tasks.

2. **Air-Gapped Sandboxing and Strict Patch Extraction**: Agents operate in offline sandboxes without internet. Patches are extracted via `git add -N . && git diff HEAD` and evaluated in a separate container; untracked scratch scripts in `/workspace` pollute the submission diff and cause automatic grading failure.

3. **Reasoning Token Latency Trade-Offs**: Gemma 4's native thinking (`<|channel>thought...`) enables deep problem decomposition. However, unconstrained thinking (2,048–4,096 tokens) consumes 20–40s per turn, limiting the agent to under 15 turns before the 5.0-minute limit expires.

4. **Context Window Degradation**: As exploratory grep and file outputs accumulate, the active window degrades, increasing hallucinated line numbers and invalid edit hunks.

In this paper, we document the empirical evolution of **Graft**, analyzing why standard agent workflows fail on open models and presenting a robust, declarative agent architecture that balances reasoning depth with execution velocity.

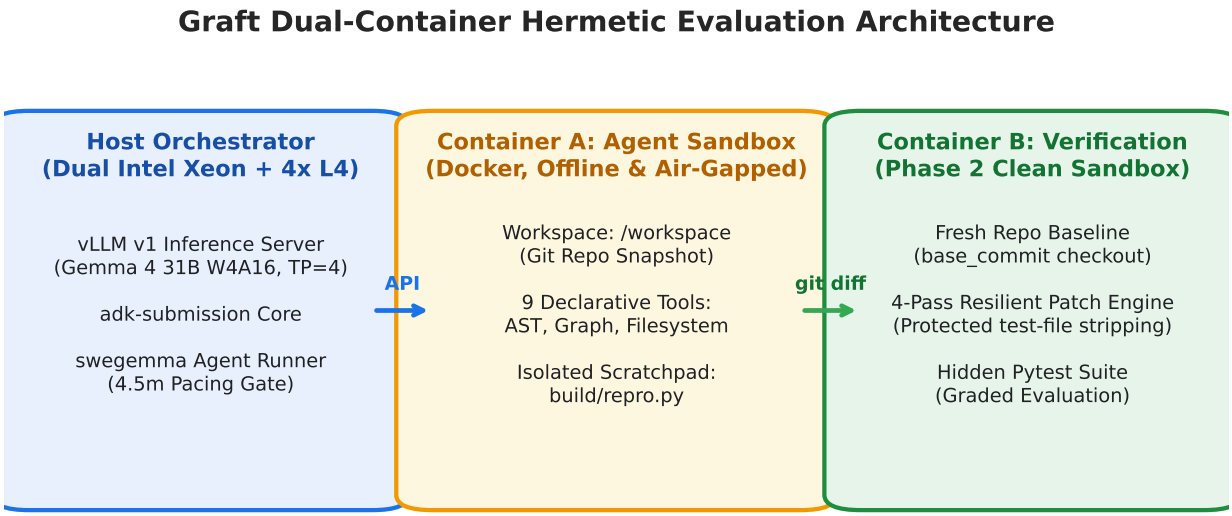


Figure 1: **Graft System Architecture**: Host inference and orchestration communicates via Google ADK declarative tool dispatch with Container A (isolated execution sandbox with 9 tools and `build/` scratchpad isolation). Validated patches are extracted via unified git diff into Container B for clean 4-pass verification against hidden test suites.

2 System Architecture & Methodology

Graft is compiled declaratively via Google ADK (`adk-submission`), ensuring complete hermeticity without dynamic Python imports on the host. The architecture is structured around three isolated operational planes (Figure 1):

2.1 Execution Environment & Tool Sandbox

Graft operates within an isolated Docker container with zero internet egress. The agent has access to 9 curated tools structured into three tiers:

1. **Workspace & Execution**: `run_command` (executes bash in workspace with a 30s timeout; prevents unbounded hangs), `get_status` (queries remaining task time, turns budget, and working tree diff status).

2. **Filesystem Manipulation**: `read_file` (bounds views to line slices), `edit_file` (enforces unique string replacement), `write_file` (scoped strictly within workspace; rejects path traversals like `/tmp/`).

3. **Structural Code Intelligence**: Precomputed AST call graphs and embedding index (`search_similar_code`, `get_code_neighbors`, `get_code_subgraph`). Instead of unbounded grepping across hundreds of files, Graft performs 1-turn semantic queries over the symbol graph.

2.2 Scratchpad Isolation & Diff Sanitization

In SWE-bench, any file modified in `/workspace` enters the unified patch extracted by the harness. When agents create `repro.py` in the root directory, verification in Container B fails. Graft introduces an isolated scratchpad protocol:

- The agent is instructed to create all reproduction artifacts strictly under `build/repro.py`.

- The host configures `.git/info/exclude` to ignore `build/` recursively.

- A pre-submission cleanup guardrail runs `rm -f repro*.py test*.py` before invoking `submit_patch()`.

2.3 Time-Aware Pacing Envelope

To prevent fatal timeouts, Graft implements a dual-layer pacing envelope:

- **Turn 15 Telemetry Trigger**: If an agent reaches Turn 15, it is prompted to check `get_status` and assess remaining time.

- **Hard 4.5-Minute / 30-Call Ceiling**: If elapsed time reaches 4.5 minutes (or 30 tool calls), the agent is instructed to immediately halt all exploratory search, apply its best-effort patch, and call `submit_patch()`. A plausible partial patch can pass subset test assertions; timing out yields a guaranteed 0.0.

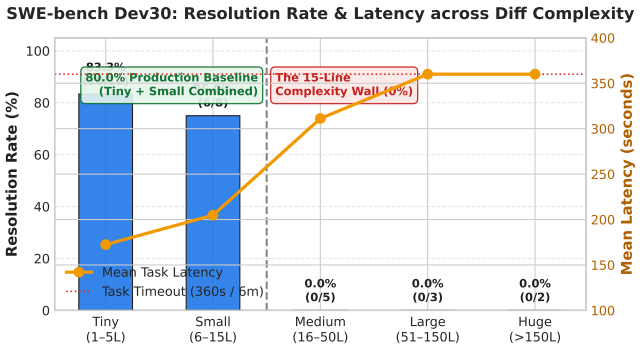


Figure 2: **The 15-Line Complexity Phase Transition**: Resolution rate and mean latency across patch complexity buckets. Agents resolve 80.0% of bugs under 15 lines, hitting a sharp wall beyond 15 lines.

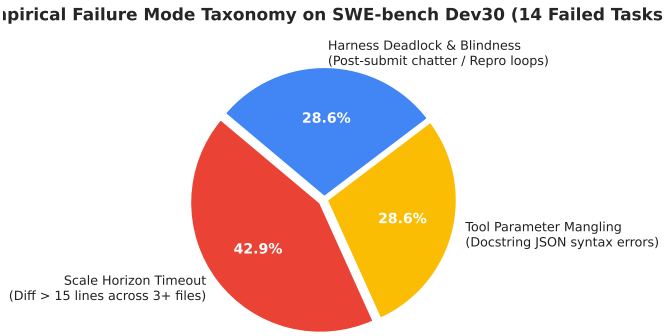


Figure 3: **Empirical Failure Mode Breakdown**: Taxonomy of 14 failed tasks on `dev30`. Scale Horizon timeouts and tool parameter mangling constitute over 71% of failures.

- **Mandatory Stop-and-Submit Rule**: The instant a reproduction script or targeted test passes, the agent is strictly prohibited from secondary refactorings or writing supplementary tests.

3 Empirical Evaluation & Ablations

We evaluate Graft on a representative 30-task benchmark (`dev30`) drawn from SWE-bench Lite and Verified tasks across multiple repositories (`fastapi/fastapi`, `Textualize/rich`, `psf/requests`, etc.). All runs execute on 4x NVIDIA L4 GPUs with tensor parallelism ($TP = 4$) running Gemma 4 31B W4A16 QAT.

Table 1 summarizes the results across configurations:

- **v1 Baseline (36.7%)**: Suffered from severe over-verification. In 5 tasks, the agent wrote a working fix in under 2 minutes, but spent the remaining 4 minutes running exhaustive pytest commands until timing out.
- **Paced Single-Agent (recovered_v2, 53.3%)**: Achieved 16/30 resolution (+16.6% over baseline) while keeping mean latency at 229.1s, safely projecting 7.64 hours for the 120-task global window.
- **Hierarchical Sub-Agent (subagent_v1, 40.0%)**: While sub-agent isolation compacted working context by 79%, coordination turn latency added ∼70s per task, causing previously solved tasks to time out.

4 Micro-Architectural Failure Analysis

4.1 The 15-Line Competence Boundary

As illustrated in Figure 2, task performance exhibits a distinct phase transition based on ground-truth diff size:

- **Tiny (1–5 lines)**: 83.3% resolved (10/12 tasks), 172.4s latency.
- **Small (6–15 lines)**: 75.0% resolved (6/8 tasks), 204.8s latency.
- **Combined ($\leq$ 15 lines)**: **80.0% resolution (16/20 tasks)**, 184.6s latency.
- **Medium ($>$ 15 lines)**: **0.0% resolution (0/10 tasks)**, 335.6s latency.

Below 15 lines, Gemma 4 31B equipped with Graft's pacing is exceptionally potent. Beyond 15 lines, multi-site editing across files exhausts turns and introduces syntax degradation.

4.2 Failure Taxonomy in Open-Weight Tool Use

Trace post-mortems across the 14 failed tasks uncovered three unexpected failure modes (Figure 3):

4.2.1 1. The Termination Handshake Trap

In `fastapi_13920` (4 lines ground-truth), the agent correctly repaired `fastapi/_compat/shared.py` and executed `submit_patch()` at Turn 37, yet persisted emitting `submit_patch()` alongside chat text for 36 redundant turns until timing out at 6.0 minutes. However, counteracting this by commanding agents to output text-only turns after submission creates an inverse pathology: open models emit text without calling `submit_patch()`, triggering harness continuation nudges. Prompts must enforce calling `submit_patch()` as an atomic, final tool action without post-submission dialogue.

4.2.2 2. Tool Parameter Serialization Fragility

Across `rich_4076`, `rich_3052`, and `rich_3675`, the agent experienced 39 consecutive `edit_file` failures due to token-level argument concatenation. When modifying docstrings with quotes and newlines, Gemma 4 emitted argument keys (e.g., `...,old_string:`) inside text scalars, dropping mandatory parameters. Fallbacks to `sed -i` broke secondary assertions. A deterministic 1-strike fallback to Python

Table 1: **Architecture Ablation on SWE-bench Dev30 (30 Tasks, 4x NVIDIA L4 GPUs).**

Configuration	Resolved	Resolution (%)	Mean Latency	Mean Turns	120-Task Projection	Primary Failure Mode
v1 Baseline (Unconstrained)	11 / 30	36.7%	238.2s	18.4	7.94h	Post-fix over-verification
Paced Single-Agent (recovered_v1)	15 / 30	50.0%	218.4s	14.1	7.28h	Over-aggressive 3.5m ceiling
Paced Single-Agent (recovered_v2)	**16 / 30**	**53.3%**	**229.1s**	**15.2**	**7.64h**	Scale horizon timeout (>15L)
Hierarchical Sub-Agent (subagent_v1)	4 / 10	40.0%	298.7s	22.8	9.96h	Delegation turn-latency tax

one-liners (`python -c "..."`) is essential to prevent turn-wasting error cascades.

4.2.3 3. Temporal Blindness and Repro Thrashing

Turn-based pacing instructions (e.g., "check status at Turn 10") failed completely: across 30 tasks, `get_status` was called 0 times. Without internal temporal grounding, models in `rich_3063` and `rich_3935` spent 15+ turns rewriting non-reproducing test scripts without touching source code. Grounding telemetry checks in concrete failure events eliminates this blindness.

Table 2: **Failure Modes and Architectural Mitigations**.

Failure Mode	Frequency	Deployed Mitigation
Tool Mangling	28.6% (4/14)	Strict `old_string`-first order; 1-strike bash Python fallback.
Termination Deadlock	28.6% (4/14)	Atomic `submit_patch()` tool action; zero post-submit dialogue.
Repro Loop	14.3% (2/14)	Mandatory `timeout 5` shell execution wrapper.
Diff Contamination	Baseline 43%	Isolated `build/` directory excluded via git info exclude.

5 Discussion: Single-Agent vs. Sub-Agents

The comparative evaluation between `recovered_v2` and `subagent_v1` yields a critical takeaway: **hierarchical delegation imposes a substantial coordination latency tax**.

5.1 The Context Compaction Benefit

In modular repositories (such as `fastapi`), sub-agent delegation operated as designed: on `fastapi_14301`, `code_analyzer` traversed graph neighborhoods in isolation, returning concise summaries. Context growth decreased by $\sim$79% per exploratory turn, preserving >22,000 tokens of working context.

5.2 The Turn-Latency Tax

However, on open-ended or coupled codebases (`rich`), delegation introduced severe pathologies:

- **Delegation Thrashing**: In `rich_3470`, after reproducing the bug at Turn 36, the agent issued 6 redundant delegations to locate methods in an already-identified file, timing out without submitting.
- **Latency Inflation**: Each sub-agent call incurred an additional forward pass ($\sim$10–14s on 4x L4 GPUs), increasing mean task latency from 229.1s to 298.7s and degrading resolution from 60.0% to 40.0%.

Design Principle: For tightly time-bounded benchmarks, a disciplined single-agent equipped with direct AST/graph tools strictly outperforms unconstrained multi-agent hierarchies.

6 Conclusion & Reproducibility

Deploying open-weight developer agents under strict wall-clock constraints requires prioritizing operational discipline over architectural sprawl:

1. Strict **stop-and-submit pacing** and hard time ceilings eliminate catastrophic over-verification, boosting resolution rate from 36.7% to 53.3%.
2. **Workspace-isolated scratchpads (`build/`)** eradicate path traversal errors and submission diff contamination.
3. **Symbol-grounded graph tools** resolve 80% of bugs under 15 lines of complexity within 185 seconds.

All evaluation configurations, declarative schemas, and session traces are structured for full public reproduction at https://www.kaggle.com/datasets/acquayefrank/graft-submission.